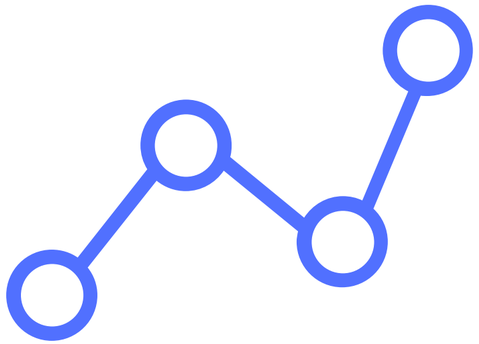
VIBE CODING 기초
5차시

RAG & LangChain 기초

RAG 개요 · LangChain 기초 실습 · RAG 활용 실습

해달 교육운영진 허진수 | 문의: 단톡방



ⓒ 2026 Jinsu Heo, All rights reserved

INDEX

5차시 수업 순서 (100분)

1 RAG 개요

2 LangChain 기초 & 실습

3 RAG 활용 실습

4 마무리 & 질의응답

ⓒ 2026 Jinsu Heo, All rights reserved

오늘의 시간표

파트	시간	내용
① RAG 개요	19:00 ~19:20	LLM의 한계, RAG 개념 · 장점, 파인튜닝 비교 → 🍵 10분 휴식
② LangChain	19:30 ~19:50	LangChain 개념, 데이터 정제, LCEL, 첫 체인 실습 → 🍵 10분 휴식
③ RAG 실습	20:00 ~20:20	미니 RAG 실습, Hybrid · Advanced · Multi-modal RAG
④ 마무리	20:20 ~20:30	질의응답, 다음 수업 안내 (6차시 Skill & MCP)

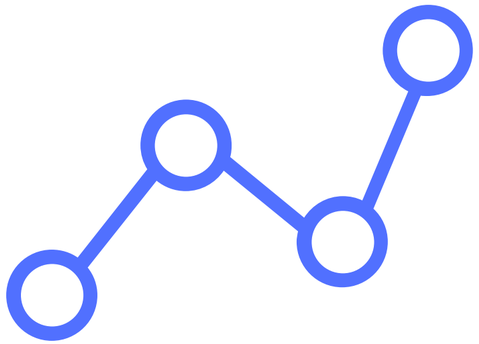
ⓒ 2026 Jinsu Heo, All rights reserved

PART 01
RAG 개요

LLM의 한계 · RAG 개념 · 장점 · 파인튜닝 비교

⏱️ 교육 시간 20분

해달 교육운영진 허진수 | 문의: 단톡방



ⓒ 2026 Jinsu Heo, All rights reserved

일반 LLM의 3가지 한계



최신성 절벽
Knowledge Cutoff

학습이 끝난 뒤의
새로운 정보를
알지 못함



폐쇄성 자물쇠
Private Data

사내 문서 · 비공개
데이터에
접근할 수 없음

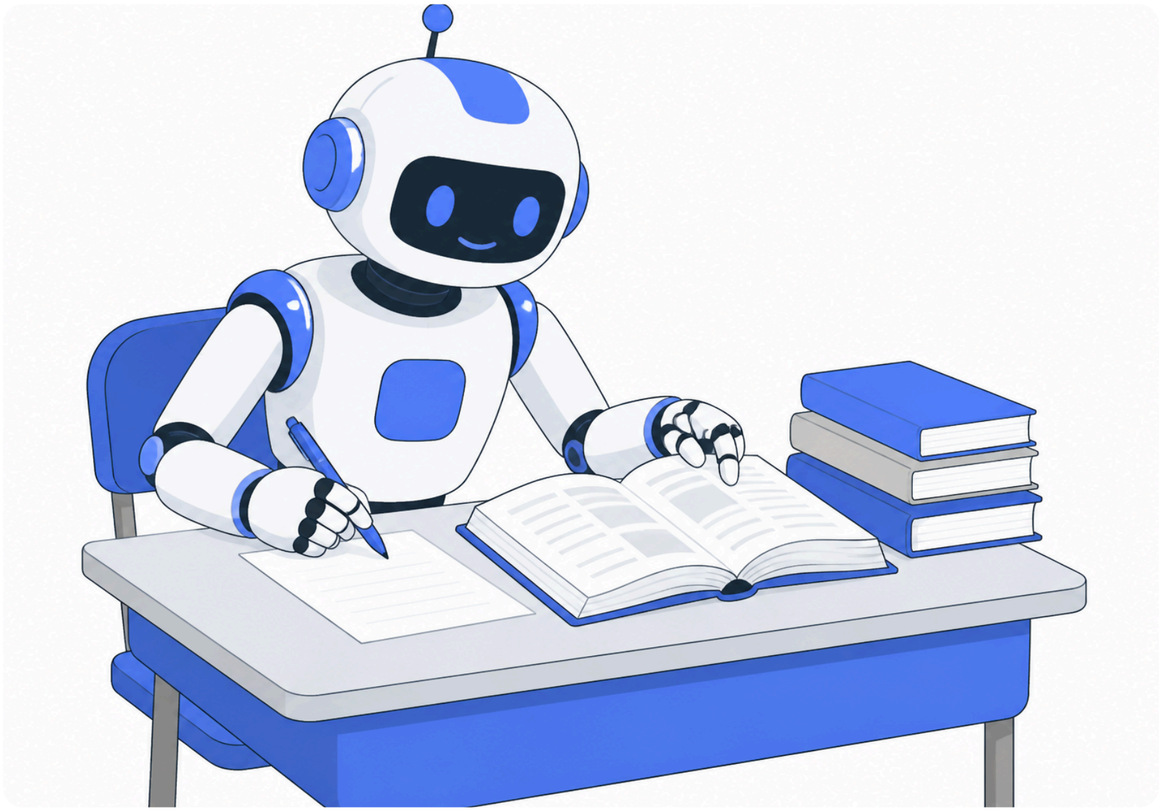


환각의 미로
Hallucination

모르는 내용도
사실처럼
그럴듯하게 꾸밈

ⓒ 2026 Jinsu Heo, All rights reserved

RAG = AI의 오픈북 시험



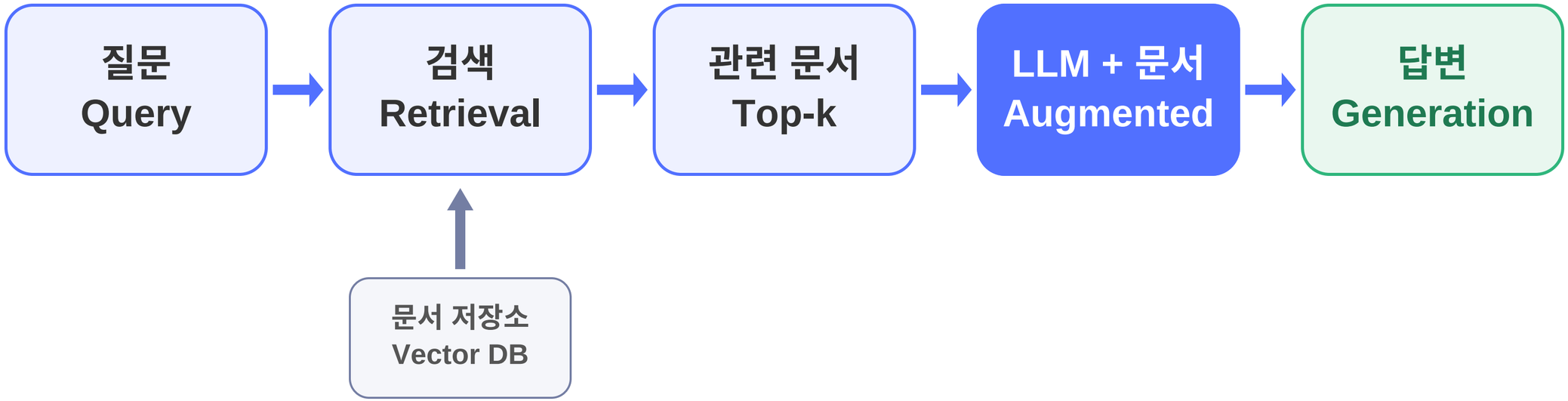
검색 증강 생성

① 질문과 관련된 문서를 먼저 검색
② 찾은 문서를 질문과 함께 LLM에 전달
③ LLM이 문서를 근거로 답변 생성

- Retrieval = 문서 검색
- Augmented = 프롬프트 보강
- Generation = 답변 생성

ⓒ 2026 Jinsu Heo, All rights reserved

RAG 동작 흐름



질문을 바로 LLM에 보내지 않고, 관련 문서를 먼저 찾아 함께 넣는 것이 핵심입니다.

ⓒ 2026 Jinsu Heo, All rights reserved

RAG의 장점 3가지



최신 · 도메인 지식 반영

재학습 없이
최신 문서를
바로 활용



환각 감소 Grounding

검색한 문문서를
근거로 답해
정확도 향상

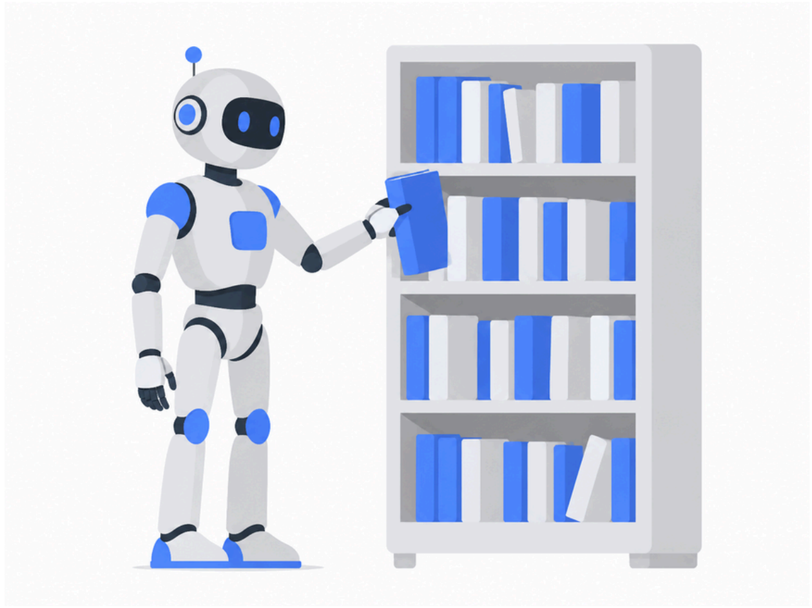


출처 제공 Citation

답변의 근거가 된
원본 문서를
함께 보여 줌

ⓒ 2026 Jinsu Heo, All rights reserved

RAG vs 파인튜닝: 찾아보기 vs 외우기



RAG

필요할 때 책을 찾아보는 학생

→ 지식이 바뀌면 문서만 교체



파인튜닝

공부해서 머릿속에 외운 학생

→ 바꾸려면 다시 학습

ⓒ 2026 Jinsu Heo, All rights reserved

RAG vs 파인튜닝 한눈에 비교

구분	RAG (검색 증강)	파인튜닝 (재학습)
바꾸는 것	**모델에 넣어 주는 지식**	모델 자체 (가중치)
지식 업데이트	**문서만 바꾸면 즉시 반영**	다시 학습해야 반영
비용 · 속도	**낮음 · 빠르게 구축**	높음 · GPU 학습 필요
적합한 경우	**자주 바뀌는 지식 · 출처가 중요**	말투 · 형식 · 전문 행동을 바꿀 때

지식이 변하면 RAG, 행동을 바꾸려면 파인튜닝

ⓒ 2026 Jinsu Heo, All rights reserved

쉬는 시간



10분 쉬어 갈게요!

19:30에 다시 시작합니다

다음 파트: LangChain 기초 & 실습
Antigravity IDE를 미리 열고 Google 계정 로그인을 확인해 주세요!

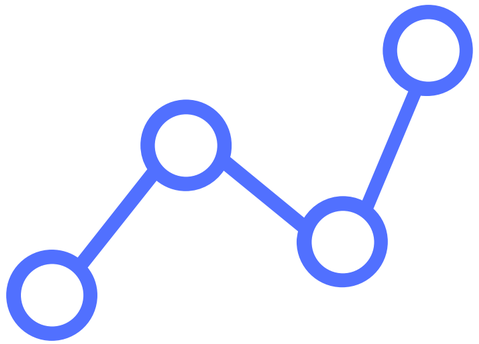
ⓒ 2026 Jinsu Heo, All rights reserved

PART 02
LANGCHAIN

LangChain 개념 · 데이터 정제 · LCEL · 첫 체인 실습

⏱️ 교육 시간 20분

해달 교육운영진 허진수 | 문의: 단톡방



ⓒ 2026 Jinsu Heo, All rights reserved

LangChain이란?



LLM 앱 조립 프레임워크

① LLM · 문서 · 도구를 사슬처럼 연결
② 모델마다 다른 호출 방식을 표준화
③ Python · JavaScript 모두 지원

- Lang = 언어 모델(LLM)
- Chain = 여러 단계를 연결
- RAG 구현에 가장 많이 쓰임

ⓒ 2026 Jinsu Heo, All rights reserved

LangChain 핵심 부품 4가지

Model (LLM)

- ✓ Gemini · GPT · Claude 호출
- ✓ 모델 교체도 한 줄이면 끝

Prompt Template

- ✓ 질문을 끼워 넣는 틀
- ✓ 변수만 바꿔 재사용

4가지 부품

Retriever

- ✓ 관련 문서를 찾아오는 검색기
- ✓ Vector DB와 연결

Output Parser

- ✓ 결과에서 필요한 부분만 추출
- ✓ 문자열 · JSON으로 변환

ⓒ 2026 Jinsu Heo, All rights reserved

데이터 정제 4단계 (Data Refinery)

1

Document Loader
문서 불러오기

PDF · CSV · 웹 페이지를
텍스트로 읽기

2

Text Splitter
잘게 나누기

문맥이 끊기지 않게
청크(Chunk)로 분할

3

Vector Store
임베딩 후 저장

청크를 숫자 벡터로
바꿔 DB에 저장

4

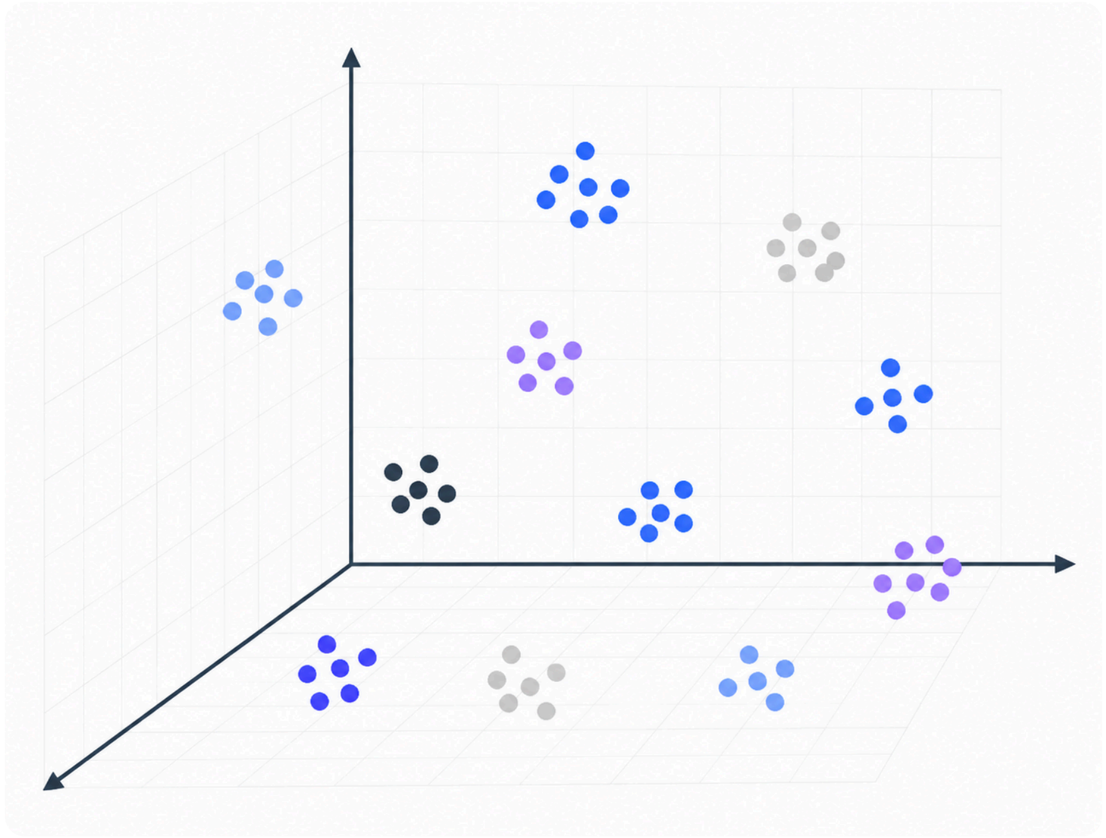
Retriever
관련 청크 검색

질문과 가까운
청크를 꺼내 오기

①~③은 미리 한 번만 준비하고, ④ Retriever는 질문할 때마다 실행됩니다.

ⓒ 2026 Jinsu Heo, All rights reserved

임베딩 (Embedding) = 의미의 좌표



문장 → 숫자 좌표

① 문장을 수백 차원의 벡터로 변환

② 뜻이 비슷하면 가까운 자리에 배치

③ 단어가 달라도 의미로 검색

예) '강아지' ↔ '반려견' = 가까움

ⓒ 2026 Jinsu Heo, All rights reserved

Vector DB = 의미로 찾는 도서관



벡터 전용 저장소

① 임베딩 벡터를 저장하는 DB

② 질문과 가장 가까운 문서를 검색

③ 수백만 건도 밀리초 단위로 탐색

- 실습용: Chroma · FAISS
- 서비스용: Pinecone · pgvector
- 오늘 실습: 메모리 저장소

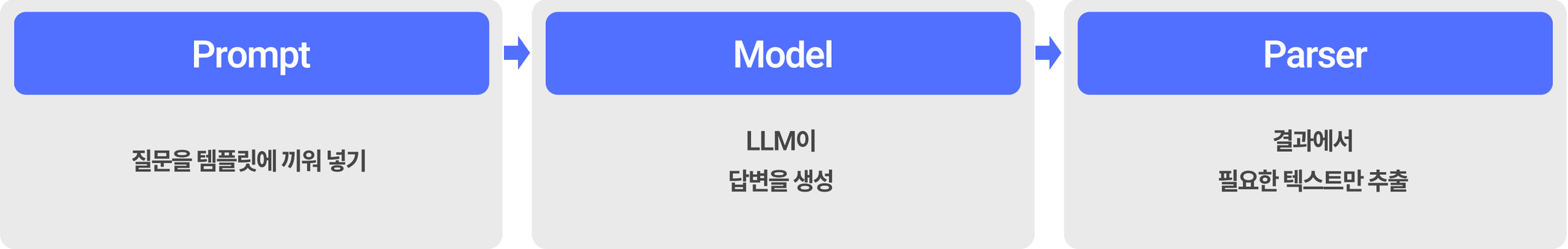
ⓒ 2026 Jinsu Heo, All rights reserved

LCEL: 파이프(|)로 잇는 체인

한 줄 문법

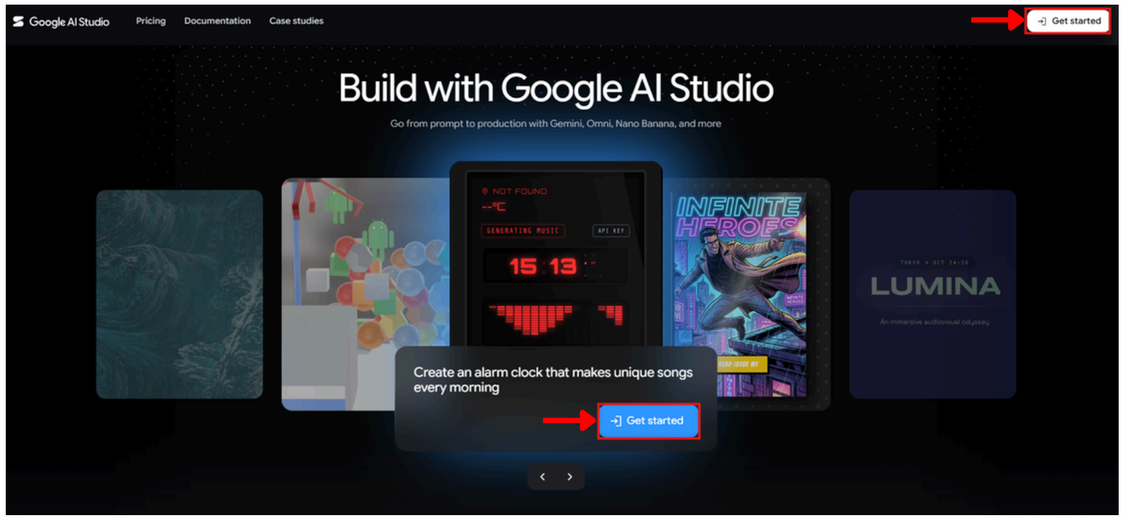
- chain = prompt | model | parser
- 왼쪽 부품의 결과가 오른쪽 부품의 입력으로 흐르는 파이프라인입니다.

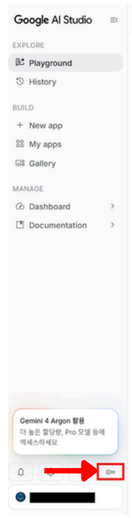
세 부품의 역할

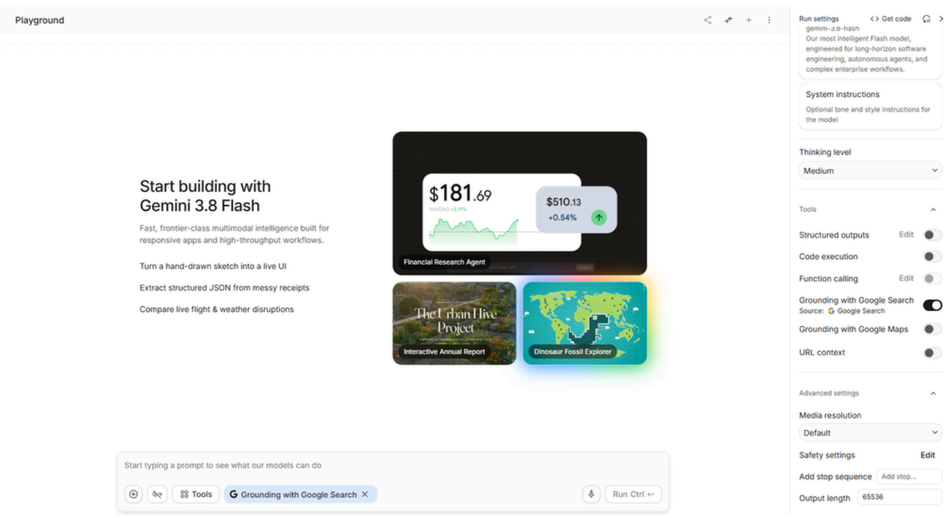


ⓒ 2026 Jinsu Heo, All rights reserved

Gemini API 무료 키 발급 ①







접속 · 로그인

① aistudio.google.com 접속

② Google 계정으로 로그인

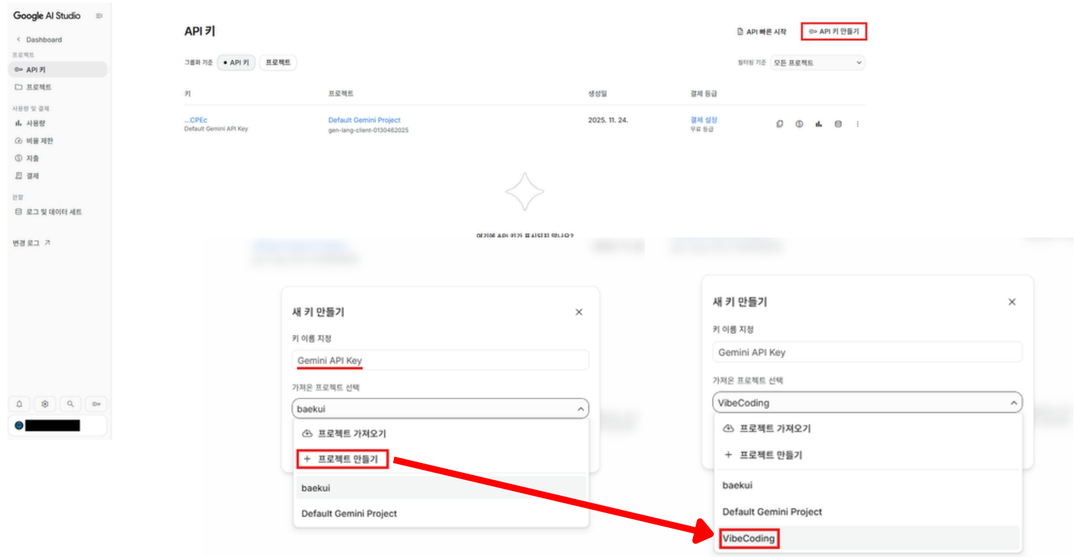
API 키 메뉴

③ 왼쪽 메뉴 Get API key 클릭

④ 이용 약관 확인 후 동의

ⓒ 2026 Jinsu Heo, All rights reserved

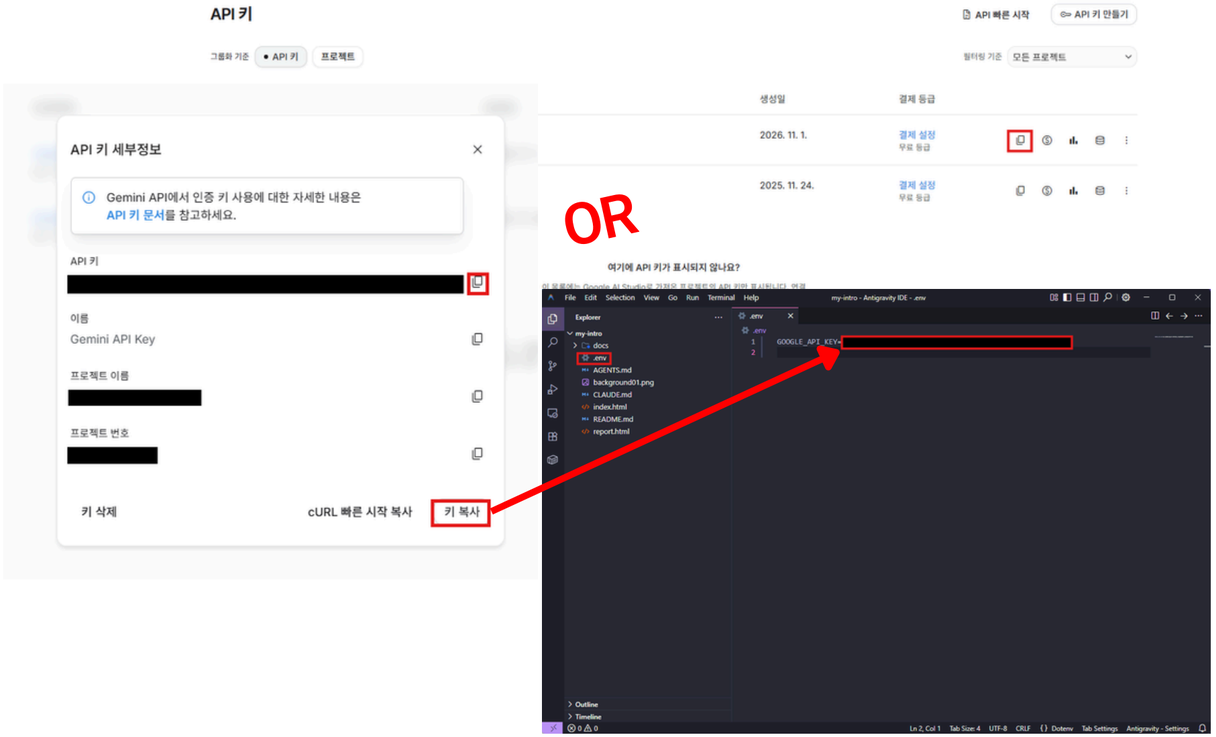
Gemini API 무료 키 발급 ②



키 만들기

⑤ Create API key 클릭

⑥ 프로젝트 선택 (없으면 새로 생성)



복사 · 보관

⑦ 생성된 키 복사

⑧ .env에 GOOGLE_API_KEY=키 저장

ⓒ 2026 Jinsu Heo, All rights reserved

Gemini 무료 키 사용 시 주의

구분	주의할 점	대처 방법
사용량 제한	횟수 제한	같은 요청 반복 금지! 429 에러가 나면 1분 뒤 다시
모델 이름	변경 잦음	"무료 Gemini Flash 계열"로 요청하고 고르기는 에이전트에게
키 보안	유출 위험	.env에만 보관, GitHub 업로드 · 단톡방 공유 금지
입력 데이터	학습 활용	개인정보 · 민감한 문서는 넣지 않기

ⓒ 2026 Jinsu Heo, All rights reserved

에이전트에게 실습 맡기는 요령

실습 방식

코드는 직접 치지 않아요!
슬라이드의 프롬프트를 Antigravity IDE 에이전트에 붙여 넣습니다.

요령 01

파일 · 모델 지정

파일명 · 모델 · 라이브러리를
프롬프트에 명시

요령 02

키는 .env에

API 키는 코드에 쓰지 않고
.env 파일로 분리

요령 03

실행까지 확인

실행 결과와 에러를
에이전트가 직접 확인

ⓒ 2026 Jinsu Heo, All rights reserved

[실습 1] 실습 프로젝트 준비

💬 Antigravity IDE 에이전트 채팅창에 입력

이 폴더에 Python 실습 환경을 만들어줘.

- 가상환경(.venv) 생성
- langchain, langchain-google-genai, python-dotenv 설치
- .env 파일에 GOOGLE_API_KEY= 자리 만들기
- .gitignore에 .env 추가

끝나면 설치된 버전을 확인해줘.

실습 순서

① 바탕화면에 rag-lab 폴더 만들기
② IDE에서 폴더 열고 프롬프트 입력
③ .env에 발급받은 키 붙여 넣기

- ⏱️ 약 5분 소요
- 설치 명령은 확인 후 허용
- API 키는 절대 공유 금지 🔒

ⓒ 2026 Jinsu Heo, All rights reserved

[실습 2] 첫 LangChain 체인 만들기

💬 Antigravity IDE 에이전트 채팅창에 입력

hello_chain.py를 만들어줘.

- LangChain LCEL 문법 사용
  (prompt | model | parser)
- 모델: 무료로 쓸 수 있는 Gemini Flash 계열
- 키는 .env의 GOOGLE_API_KEY에서 읽기
- 주제를 받아 초보자용 3줄 설명 출력

만든 뒤 '임베딩' 주제로 실행해줘.

실습 순서

① 프롬프트를 복사해 입력하기
② 만들어진 코드에서 | 찾아보기
③ 터미널의 실행 결과 확인하기

- ⏱️ 약 5분 소요
- 요청은 한 번만! (무료 사용량)
- 에러는 그대로 붙여 넣기

ⓒ 2026 Jinsu Heo, All rights reserved

[실습 2] 결과 확인하기

📄 hello_chain.py (에이전트가 만든 코드 예시)

```
from dotenv import load_dotenv
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser
from langchain_google_genai import ChatGoogleGenerativeAI

load_dotenv()
prompt = ChatPromptTemplate.from_template(
    "{topic}을(를) 초보자에게 3줄로 설명해줘.")
model = ChatGoogleGenerativeAI(model="gemini-flash-latest")
parser = StrOutputParser()

chain = prompt | model | parser
print(chain.invoke({"topic": "임베딩"}))
```

결과 보는 법

① 터미널에 3줄 설명이 나오면 성공

② 코드에서 chain = … 한 줄 찾기

③ topic만 바꿔 다시 실행해 보기

핵심은 chain = prompt | model | parser 한 줄입니다.
나머지는 키를 불러오고 결과를 출력하는 코드예요.

ⓒ 2026 Jinsu Heo, All rights reserved

쉬는 시간



수고했어요! 10분 휴식

20:00에 다시 시작합니다

다음 파트: RAG 활용 실습
실습 1에서 만든 rag-lab 폴더를 그대로 사용합니다!

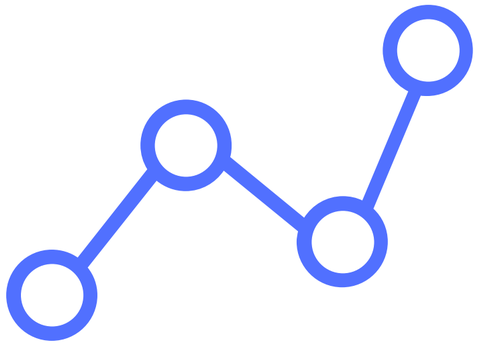
ⓒ 2026 Jinsu Heo, All rights reserved

PART 03
RAG 활용 실습

미니 RAG 실습 · Hybrid · Advanced · Multi-modal RAG

⏱️ 교육 시간 20분

해달 교육운영진 허진수 | 문의: 단톡방



ⓒ 2026 Jinsu Heo, All rights reserved

실습 시나리오: 미니 리뷰 챗봇



무선 이어폰 리뷰 챗봇

① 가상 리뷰 10개를 파일로 만들기
② 리뷰를 임베딩해 저장하기
③ 근거 리뷰와 함께 답하기

- 리뷰 10개 · 검색 상위 3개
- 저장소: 메모리(InMemory)
- 모델: Gemini 무료 키

ⓒ 2026 Jinsu Heo, All rights reserved

[실습 3] 리뷰 데이터 만들기

💬 Antigravity IDE 에이전트 채팅창에 입력

reviews.txt 파일을 만들어줘.

- 가상의 무선 이어폰 리뷰 10개
- 한 줄에 하나씩: 별점 | 작성자 | 내용
- 운동 · 배터리 · 음질 · 착용감 · 방수 주제가 골고루 섞이게
- 작성자는 김*수처럼 가려서

실제 개인정보는 넣지 마.

실습 순서

① 프롬프트를 복사해 입력하기
② reviews.txt를 열어 내용 확인
③ 리뷰 1~2개는 직접 고쳐 보기

- ⏱️ 약 3분 소요
- 이 파일이 RAG의 '참고서'
- Gemini 키는 아직 사용 안 함

ⓒ 2026 Jinsu Heo, All rights reserved

[실습 4] 미니 RAG 만들기

💬 **Antigravity IDE 에이전트 채팅창에 입력**

LangChain으로 rag.py를 만들어줘.

1. reviews.txt를 줄 단위 문서로 불러오기
2. Gemini 임베딩 모델로 벡터 변환
3. InMemoryVectorStore에 저장
4. 질문과 가장 가까운 리뷰 3개 검색
5. 검색된 리뷰만 근거로 답변
   (리뷰에 없으면 '알 수 없다'고 답하기)
6. 답변 아래에 근거 리뷰 출력

코드 속 RAG 흐름

Loader → Embedding → Vector Store
→ Retriever → Prompt | Model | Parser

확인 포인트

프롬프트의 '알 수 없다' 규칙이
환각을 막는 가드레일입니다.

ⓒ 2026 Jinsu Heo, All rights reserved

[실습 5] 질문하고 출처 확인하기

💬 Antigravity IDE 에이전트 채팅창에 입력

rag.py를 실행해서
아래 두 질문을 차례로 해줘.

Q1. 운동할 때 써도 괜찮아?
Q2. 이 이어폰 가격은 얼마야?

각 답변과 근거 리뷰를 보여줘.

예상 결과

Q1 → "땀에 강하고 잘 빠지지 않는다는 리뷰가 많아요."
📎 근거: ★5 김*수 · ★4 이*영

Q2 → "리뷰에 가격 정보가 없어요."
✅ 모르면 모른다고 하면 성공!

ⓒ 2026 Jinsu Heo, All rights reserved

RAG 파이프라인 고도화 3가지



Streaming
체감 속도 개선

답변을 완성 전부터
한 글자씩 보여 주기



Score Filtering
정확도 향상

유사도 점수가 낮은
문서는 미리 버리기



Re-ranking
답변 품질 향상

검색 결과 순서를
AI가 다시 매기기

ⓒ 2026 Jinsu Heo, All rights reserved

Hybrid Search

두 검색 결과를 RRF(순위 합치기)로 섞으면 Hybrid Search





Dense 검색

1. 의미가 비슷한 문장 찾기
2. 임베딩 벡터 거리 비교

Sparse 검색

1. 단어가 일치하는 문서 찾기
2. BM25 키워드 점수

ⓒ 2026 Jinsu Heo, All rights reserved

Advanced RAG ① 문맥 손실과 해결책



문맥 손실

문서를 자르면 '그 제품', '이 기능'처럼 주어가 사라져
검색해도 무엇에 대한 내용인지 알 수 없는 조각이 생깁니다.

✔ 문제

Context Loss

주어가 빠지고 맥락이
잘려 나간 조각

✔ 해결 01

Contextual Retrieval

청크마다 문서 요약을
앞에 붙여서 저장

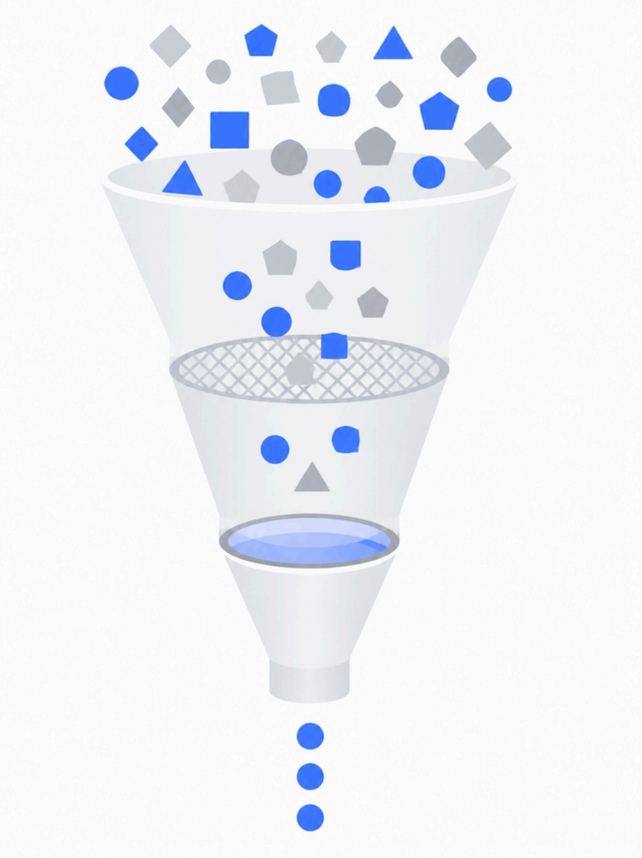
✔ 해결 02

Late Chunking

문서 전체를 먼저 임베딩하고
그 뒤에 잘게 분할

ⓒ 2026 Jinsu Heo, All rights reserved

Advanced RAG ② 검색 깔때기



① 1차 검색 (Initial Retrieval)

후보를 넉넉하게 20~50개 찾기 (Recall 우선)

② MMR (Maximal Marginal Relevance)

비슷한 문서는 덜어내고 다양한 관점 남기기

③ 리랭킹 (Re-ranking)

Cross-Encoder로 다시 채점해 Top 3~5만 사용

ⓒ 2026 Jinsu Heo, All rights reserved

Multi-modal RAG란?



글 너머까지 검색

① 이미지 · 표 · 차트도 검색 대상

② 질문과 가까운 이미지를 함께 찾기

③ Vision LLM이 이미지를 보고 답변

- 제품 사진 속 색상 질문
- 보고서 속 차트 해석
- 매뉴얼 도식 설명

ⓒ 2026 Jinsu Heo, All rights reserved

Multi-modal RAG 구현 2가지

구분	① CLIP 방식	② Vision LLM 방식
핵심 아이디어	이미지를 바로 벡터로	이미지를 글로 요약한 뒤 벡터로
흐름	CLIP → 공유 벡터 공간	Vision LLM → 설명문 → 텍스트 임베딩
장점	비슷한 이미지 찾기	기존 텍스트 RAG를 그대로 재사용
주의점	텍스트와 공간 어긋남	요약할 때 세부 정보가 빠질 수 있음

시각적으로 비슷한 것을 찾으면 ①, 표·도면 속 정보를 찾으면 ②

ⓒ 2026 Jinsu Heo, All rights reserved

부록: End-to-End 파이프라인



오늘 만든 미니 RAG와 같은 구조에, 저장소와 화면만 서비스용으로 바뀝니다.

ⓒ 2026 Jinsu Heo, All rights reserved

부록: 실전 기술 스택 예시

실전 스택

프론트 · 백엔드

- Next.js (API Routes)
- 화면과 서버를 한 프로젝트로

Vector DB

- Pinecone (완전 관리형)
- 벡터 검색을 밀리초 단위로

RAG 프레임워크

- LangChain.js · LCEL
- 체인과 데이터 파이프라인 관리

LLM · DB

- Gemini Flash 계열 · Supabase
- 빠른 추론 + 메타데이터 저장

ⓒ 2026 Jinsu Heo, All rights reserved

자주 겪는 문제



대부분의 원인

코드보다 환경 설정에서 막히는 경우가 대부분입니다.
아래 세 가지만 확인하면 대부분 해결됩니다!

문제 01

API 키 오류

.env 위치와 변수명 확인
GOOGLE_API_KEY=키값

문제 02

429 요청 초과

무료 한도를 잠시 초과
1분 기다린 뒤 다시 실행

문제 03

ModuleNotFound

가상환경(venv) 켜기
에이전트에게 재설치 요청

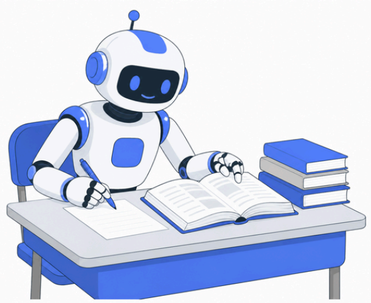
ⓒ 2026 Jinsu Heo, All rights reserved

실습 완료 체크리스트

확인 항목	구분	확인 방법
API 키	필수	□ .env에 GOOGLE_API_KEY를 저장하고 실행 시 오류가 없는지 확인
LCEL 체인	필수	□ prompt | model | parser 체인이 답변을 출력하는지 확인
미니 RAG	필수	□ 리뷰를 근거로 답하고 출처 리뷰를 함께 보여 주는지 확인
환각 방지	선택	□ 리뷰에 없는 질문에 '모른다'고 답하는지 확인

ⓒ 2026 Jinsu Heo, All rights reserved

오늘 배운 것



RAG
= = 오픈북 시험

관련 문서를 먼저 찾고
그 문서를 근거로
답하게 하기!



LangChain
+ LCEL

prompt | model | parser
부품을 파이프로
이어 붙이기!



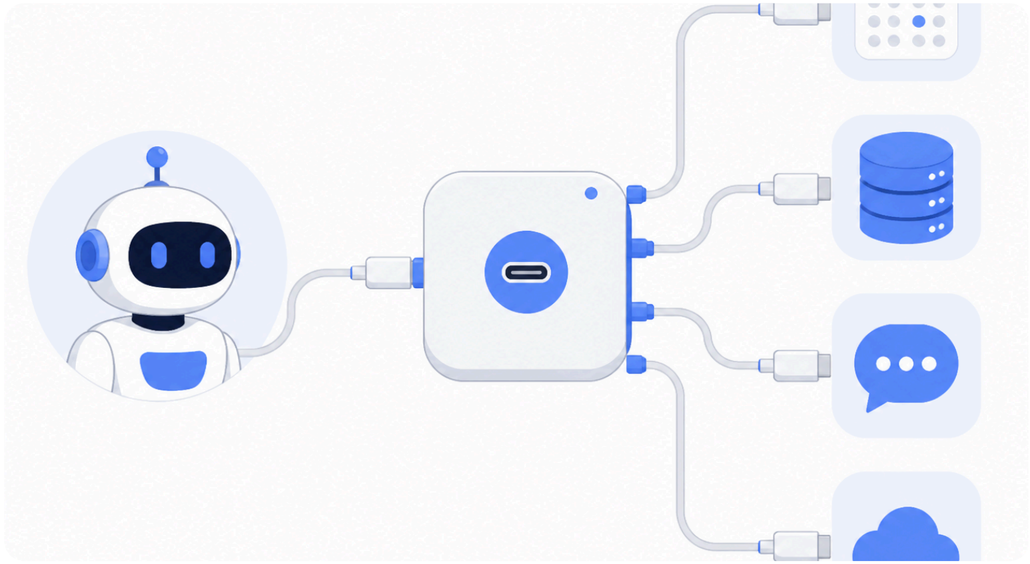
미니 RAG
직접 만들기

리뷰 검색 → 답변
→ 출처 확인까지
한 번에!

ⓒ 2026 Jinsu Heo, All rights reserved

다음 수업: 6차시 Skill & MCP





Skill

AI에게 반복 작업 요령을 가르치는 설명서

한 번 만들면 계속 재사용

MCP

AI를 외부 도구 · 데이터와 잇는 표준 연결선

Notion · GitHub · DB 등과 연결

ⓒ 2026 Jinsu Heo, All rights reserved

질문과 답변



궁금한 점, 무엇이든 물어보세요!

RAG · LangChain, 무엇이든 OK

실습을 다 끝내지 못했다면 미니 RAG(실습 4)까지만 먼저 완성해 보세요.
에러 화면은 캡처해서 단톡방에 올려 주면 함께 해결합니다!

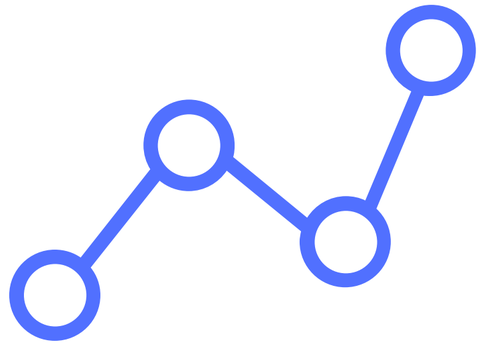
ⓒ 2026 Jinsu Heo, All rights reserved

THANK YOU!!

6차시에 만나요~!

오늘 만든 미니 리뷰 챗봇 실행 화면, 단톡방에 공유해 주세요!

해달 교육운영진 허진수 | 문의: 단톡방 / zintn11@gmail.com



ⓒ 2026 Jinsu Heo, All rights reserved